

Book Store Analysis

Kousik Chakraborty

Questions

Basic Questions:

1. Retrieve all books in the "Fiction" genre.

Query: `SELECT *FROM BOOKS WHERE GENRE='Fiction';`

	book_id [PK] integer	title character varying (100)	author character varying (100)	genre character varying (50)	published_year integer	price numeric (10,2)	stock integer
1	4	Customizable 24hour product	Christopher Andrews	Fiction	2020	43.52	8
2	22	Multi-layered optimizing migration	Wesley Escobar	Fiction	1908	39.23	78
3	28	Expanded analyzing portal	Lisa Coffey	Fiction	1941	37.51	79
4	29	Quality-focused multi-tasking challenge	Katrina Underwood	Fiction	1905	31.12	100
5	31	Implemented encompassing conglomerati...	Melissa Taylor	Fiction	2010	21.23	44

2. Find books published after the year 1950.

Query: `SELECT *FROM BOOKS WHERE PUBLISHED_YEAR>1950;`

	book_id [PK] integer	title character varying (100)	author character varying (100)	genre character varying (50)	published_year integer	price numeric (10,2)	stock integer
1	2	Persevering reciprocal knowledge user	Mario Moore	Fantasy	1971	35.80	19
2	4	Customizable 24hour product	Christopher Andrews	Fiction	2020	43.52	8
3	5	Adaptive 5thgeneration encoding	Juan Miller	Fantasy	1956	10.95	16
4	6	Advanced encompassing implementation	Bryan Morgan	Biography	1985	6.56	2
5	8	Persistent local encoding	Troy Cox	Science Fiction	2019	48.99	84

3. List all customers from the Canada.

Query: `SELECT *FROM CUSTOMERS WHERE COUNTRY='Canada';`

	customer_id [PK] integer	name character varying (100)	email character varying (100)	phone character varying (15)	city character varying (50)	country character varying (150)
1	38	Nicholas Harris	christine93@perkins.com	1234567928	Davistown	Canada
2	415	James Ramirez	robert54@hall.com	1234568305	Maxwelltown	Canada
3	468	David Hart	stokesrebecca@gmail.com	1234568358	Thompsonfurt	Canada

4. Show orders placed in November 2023.

Query: `SELECT *FROM ORDERS WHERE ORDER_DATE BETWEEN '2023-11-1' AND '2023-11-30';`

	order_id [PK] integer	customer_id integer	book_id integer	order_date date	quantity integer	total_amount numeric (10,2)
1	4	433	343	2023-11-25	7	301.21
2	19	496	60	2023-11-17	9	316.26
3	75	291	375	2023-11-30	5	170.75
4	132	469	333	2023-11-22	7	194.32
5	137	474	471	2023-11-25	8	363.04
6	163	207	384	2023-11-23	3	101.76

5. Retrieve the total stock of books available.

Query: `SELECT SUM(STOCK) AS TOTAL_STOCK FROM BOOKS;`

	total_stock bigint
1	25056

6. Find the details of the most expensive book.

Query: `SELECT *FROM BOOKS ORDER BY PRICE DESC LIMIT 1;`

	book_id [PK] integer	title character varying (100)	author character varying (100)	genre character varying (50)	published_year integer	price numeric (10,2)	stock integer
1	340	Proactive system-worthy orchestration	Robert Scott	Mystery	1907	49.98	88

7. Show all customers who ordered more than 1 quantity of a book.

Query: `SELECT *FROM ORDERS WHERE QUANTITY>1;`

	order_id [PK] integer	customer_id integer	book_id integer	order_date date	quantity integer	total_amount numeric (10,2)
1	1	84	169	2023-05-26	8	188.56
2	2	137	301	2023-01-23	10	216.60
3	3	216	261	2024-05-27	6	85.50
4	4	433	343	2023-11-25	7	301.21
5	5	14	431	2023-07-26	7	136.36

8. Retrieve all orders where the total amount exceeds \$20.

Query: `SELECT *FROM ORDERS WHERE TOTAL_AMOUNT>20;`

	order_id [PK] integer	customer_id integer	book_id integer	order_date date	quantity integer	total_amount numeric (10,2)
1	1	84	169	2023-05-26	8	188.56
2	2	137	301	2023-01-23	10	216.60
3	3	216	261	2024-05-27	6	85.50
4	4	433	343	2023-11-25	7	301.21
5	5	14	431	2023-07-26	7	136.36
6	6	439	119	2024-10-11	5	249.40

9. List all genres available in the Books table.

Query: SELECT DISTINCT GENRE FROM BOOKS;

	genre character varying (50)
1	Romance
2	Biography
3	Mystery
4	Fantasy
5	Fiction
6	Non-Fiction
7	Science Fiction

10. Find the book with the lowest stock.

Query: SELECT *FROM BOOKS ORDER BY STOCK LIMIT 1;

	book_id [PK] integer	title character varying (100)	author character varying (100)	genre character varying (50)	published_year integer	price numeric (10,2)	stock integer
1	44	Networked systemic implementation	Ryan Frank	Science Fiction	1965	13.55	0

11. Calculate the total revenue generated from all orders.

Query: SELECT SUM(TOTAL_AMOUNT) AS TOTAL_REVENUE FROM ORDERS;

	total_revenue numeric
1	75628.66

Advance Questions:

1. Retrieve the total number of books sold for each genre.

Query: `SELECT GENRE,SUM(QUANTITY) AS TOTAL_BOOKS FROM ORDERS JOIN BOOKS ON ORDERS.BOOK_ID=BOOKS.BOOK_ID GROUP BY GENRE;`

	genre character varying (50) 🔒	total_books bigint 🔒
1	Romance	439
2	Biography	285
3	Mystery	504
4	Fantasy	446
5	Fiction	225
6	Non-Fiction	351
7	Science Fiction	447

2. Find the average price of books in the "Fantasy" genre.

Query: `SELECT AVG(PRICE)AS AVERAGE_BOOK_PRICE FROM BOOKS WHERE GENRE='Fantasy';`

	average_book_price numeric 🔒
1	25.9816901408450704

3. List customers who have placed at least 2 orders.

Query: `SELECT O.CUSTOMER_ID,C.NAME,COUNT(O.ORDER_ID)
AS ORDER_COUNT FROM ORDERS O JOIN CUSTOMERS C ON
O.CUSTOMER_ID=C.CUSTOMER_ID`

GROUP BY O.CUSTOMER_ID,C.NAME HAVING COUNT(ORDER_ID)>=2;

	customer_id integer	name character varying (100)	order_count bigint
1	225	Christopher Mccullough	2
2	418	Kiara Blankenship MD	3
3	322	William Cameron	3
4	325	Emily Vargas	4
5	376	Justin Donaldson	2
6	486	Melanie Kelly	2
7	461	Crystal Pierce	3

4. Find the most frequently ordered book.

Query: SELECT O.BOOK_ID,B.TITLE,COUNT(O.ORDER_ID) AS ORDER_COUNT FROM ORDERS O JOIN BOOKS B ON O.BOOK_ID=B.BOOK_ID GROUP BY O.BOOK_ID,B.TITLE ORDER BY COUNT(O.ORDER_ID) DESC LIMIT 1;

	book_id integer	title character varying (100)	order_count bigint
1	88	Robust tangible hardware	4

5. Show the top 3 most expensive books of 'Fantasy' Genre.

Query: SELECT *FROM BOOKS WHERE GENRE='Fantasy' ORDER BY PRICE DESC LIMIT 3;

	book_id [PK] integer	title character varying (100)	author character varying (100)	genre character varying (50)	published_year integer	price numeric (10,2)	stock integer
1	240	Stand-alone content-based hub	Lisa Ellis	Fantasy	1957	49.90	41
2	462	Innovative 3rdgeneration database	Allison Contreras	Fantasy	1988	49.23	62
3	238	Optimized even-keeled analyzer	Sherri Griffith	Fantasy	1975	48.97	72

6. Retrieve the total quantity of books sold by each author.

Query: SELECT b.author, SUM(o.quantity) AS Total_Books_Sold
FROM orders o
JOIN books b ON o.book_id=b.book_id
GROUP BY b.Author;

	author character varying (100) 🔒	total_books_sold bigint 🔒
1	Jared Cortez	10
2	Tracy Parker	11
3	Taylor Wang	9
4	Cathy Knight	6
5	Bianca Matthews	3
6	Douglas Malone	6
7	James Alvarado	9
8	Betty Cross	6

7. List the cities where customers who spent over \$30 are located.

Query: SELECT DISTINCT c.city, total_amount
FROM orders o
JOIN customers c ON o.customer_id=c.customer_id
WHERE o.total_amount > 30;

	city character varying (50) 🔒	total_amount numeric (10,2) 🔒
1	Taylorfurt	189.45
2	Leeport	141.39
3	Port Jasonview	149.12
4	Port Aaronstad	145.44
5	Matthewfurt	328.50
6	Angelaside	42.19
7	Lindaburgh	325.92
8	Stephanieberg	156.60

8. Find the customer who spent the most on orders.

Query: SELECT c.customer_id, c.name, SUM(o.total_amount) AS Total_Spent
FROM orders o
JOIN customers c ON o.customer_id=c.customer_id
GROUP BY c.customer_id, c.name
ORDER BY Total_spent Desc LIMIT 1;

	customer_id [PK] integer	name character varying (100)	total_spent numeric
1	457	Kim Turner	1398.90

9. Calculate the stock remaining after fulfilling all orders.

Query:

```
SELECT b.book_id, b.title, b.stock, COALESCE(SUM(o.quantity),0) AS Order_quantity,
       b.stock- COALESCE(SUM(o.quantity),0) AS Remaining_Quantity
```

```
FROM books b
```

```
LEFT JOIN orders o ON b.book_id=o.book_id
```

```
GROUP BY b.book_id ORDER BY b.book_id;
```

	book_id [PK] integer	title character varying (100)	stock integer	order_quantity bigint	remaining_quantity bigint
1	1	Configurable modular throughput	100	3	97
2	2	Persevering reciprocal knowledge user	19	0	19
3	3	Streamlined coherent initiative	27	5	22
4	4	Customizable 24hour product	8	0	8
5	5	Adaptive 5thgeneration encoding	16	8	8
6	6	Advanced encompassing implementation	2	0	2
7	7	Open-architected exuding structure	95	5	90

